40 Days of LeetCode Challenge

40 days, 40 problems
DAY :1
LEETCODE PROBLEM NO. 1342 :Number of Steps to Reduce a Number to Zero
Problem Link : https://leetcode.com/problems/number-of-steps-to-reduce-a-number-to-zero/description/
Easy · Bit Manipulation / Simulation
STEP BY STEP SOLUTION:-

Step 1. Understand the question :-

We are given a non-negative integer num. Our job is to make it equal to 0 by repeatedly following these two rules:

- Rule 1: If the number is even, divide it by 2.
- Rule 2: If the number is odd, subtract 1 from it.

Every time we perform either operation, we count one step .
Our task is to return the total number of steps needed to reach zero.

We're not counting how many divisions happen or how many subtractions happen separately.

We're counting every operation until the number becomes zero.

Step 2. Translating it into simpler language with examples :-

Imagine num is a number on a board. We can only do two things:

- If it's divisible by 2, cut it in half.
- Otherwise, decrease it by 1

Keep repeating until we reach zero, counting each move.

Let's understand the examples:-

Example 1:
Input: num = 14
Output: 6
Explanation:
Step 1) 14 is even; divide by 2 and obtain 7.
Step 2) 7 is odd; subtract 1 and obtain 6.
Step 3) 6 is even; divide by 2 and obtain 3.
Step 4) 3 is odd; subtract 1 and obtain 2.
Step 5) 2 is even; divide by 2 and obtain 1.
Step 6) 1 is odd; subtract 1 and obtain 0.

Input: num = 14

14 ÷ 2 = 7; 7− 1 = 6; 6 ÷ 2 = 3; 3 − 1 = 2; 2 ÷ 2 = 1; 1− 1= 0

Total operations :- 6 steps

Step	Current number	Operation	New number
1	14	Divide by 2	7
2	7	Subtract 1	6
3	6	Divide by 2	3
4	3	Subtract 1	2
5	2	Divide by 2	1
6	1	Subtract 1	0

Example 2:
Input: num = 8
Output: 4
Explanation:
Step 1) 8 is even; divide by 2 and obtain 4.
Step 2) 4 is even; divide by 2 and obtain 2.
Step 3) 2 is even; divide by 2 and obtain 1.
Step 4) 1 is odd; subtract 1 and obtain 0.

Input: num = 8

8 ÷ 2 = 4; 4 ÷ 2 = 2; 2 ÷ 2 = 1; 1 − 1= 0

Total operations :- 4 steps

Step	Current number	Operation	New number
1	8	Divide by 2	4
2	4	Divide by 2	2
3	2	Divide by 2	1
4	1	Subtract 1	0

The pattern is simple:

check the current number,

perform the correct operation,

count one step, and repeat until zero.

Step 3. Identify the pattern

There are three key observations.

1. Pattern A: Every operation depends on parity.

Parity means whether a number is even or odd.

- Even: num % 2 == 0
- Odd: num % 2 != 0

The % operator returns the remainder after division. For example, 6 % 2 gives 0, while 7 % 2 gives 1.

2. Pattern B: The same decision repeats.
    - After changing the number, we must check its new value again.
    - For example, 6 becomes 3, so our next operation changes from division to subtraction.
3. Pattern C: Zero is the stopping condition.
    - We repeat the process until num == 0.
    - This is a classic problem involving a while loop, an if-else conditional, and a counter.

Step 4. Build the logic

Algorithm :-

1. Start with the given number and initialize a counter to zero.
2. While the number is greater than zero:
    - If the number is even, divide it by two.
    - Otherwise, subtract one.
    - Increase the counter by one.
3. When the number reaches zero, return the counter.

Pseudocode :-

```
FUNCTION numberOfSteps(num):
  steps = 0

  WHILE num > 0:
    IF num is even:
        num = num / 2
    ELSE:
        num = num - 1
        steps = steps + 1
  RETURN steps
```

NOTE:-

Why do we increment steps after the conditional?

Because both branches perform exactly one operation. Placing the counter increment outside the

if-else ensures that each operation is counted exactly once.

Step 5. Code in Java

```
class Solution {
   public int numberOfSteps(int num) {
      int steps = 0;

      while (num > 0) {
         if (num % 2 == 0) {
            num = num / 2;
         } else {
            num = num - 1;
         }

         steps++;
      }

      return steps;
   }
}
```

How to submit

1. Open LeetCode Problem 1342.
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Code	Explanation
int steps = 0;	Creates a counter to track the number of operations.
while (num > 0)	Continues looping as long as the number is not zero.
num % 2 == 0	Checks whether the number is even.
num = num / 2;	Divides an even number by two.
num = num - 1;	Subtracts one from an odd number.
steps++;	Increments the counter by one after each operation.
return steps;	Returns the total number of operations after the loop ends.

Dry run: num = 14

Initially, steps = 0.

Iteration	num before	Condition	num after	steps after
1	14	Even	7	1
2	7	Odd	6	2
3	6	Even	3	3
4	3	Odd	2	4
5	2	Even	1	5
6	1	Odd	0	6

Now num > 0 is false, so the loop stops and returns 6.

NOTE:-

What if the input is zero?

For num = 0, the while condition is false immediately. The loop never executes, and the method returns 0.

This is an important edge case: our solution handles zero without needing a separate if statement.

7. Analyze complexity and optimization

Time complexity: (O(log n))

1. When the number is even, we halve it.
2. When it is odd, subtracting one makes it even, so the next operation halves it.
3. Thus, every one or two operations reduce a positive number by roughly half.
4. The number of operations therefore grows logarithmically with the input size.

Auxiliary space complexity: (O(1))

1. We use only two integer variables, num and steps.
2. We do not create arrays, collections, or recursive calls, so the extra memory stays constant.

NOTE:-

Can we optimize it?

This iterative solution is already efficient. We could use bitwise operations to check parity or halve the number, but t hat would not improve the asymptotic time or space complexity.

For learning, the arithmetic version is clearer and easier to explain.

SCREENSHOT:-

